ИННОТЕХ
ГРУППА КОМПАНИЙ Т1

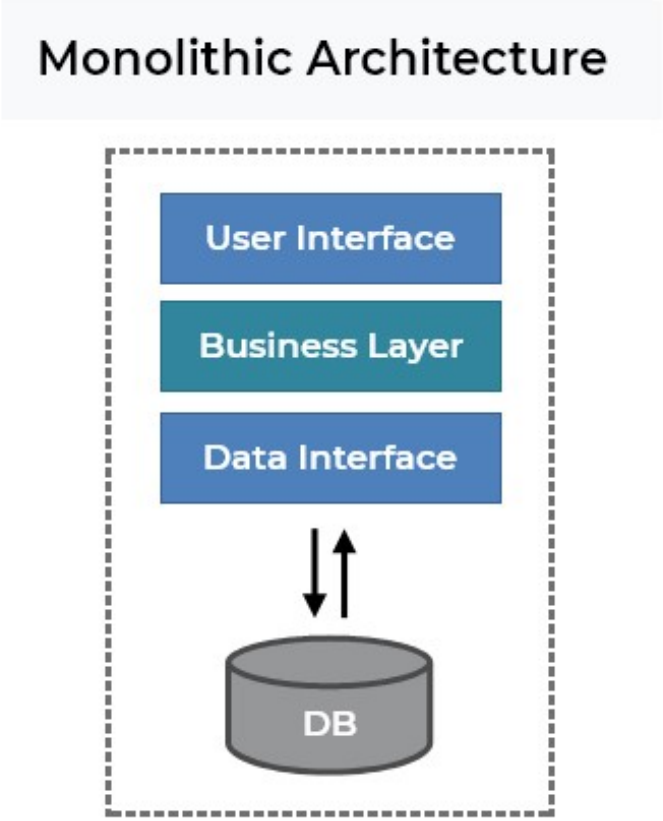
ВВЕДЕНИЕ В МИКРОСЕРВИСНУЮ АРХИТЕКТУРУ

План

- Монолитные приложения.
- Микросервисная архитектура.
- Как понять что у нас микросервисы?
- Распределенные системы.
- Критерии выбора Монолитной или Микросервисной Архитектуры.

Концепция монолитного приложения

Различные компоненты приложения объединяются в одну программу на одной платформе.

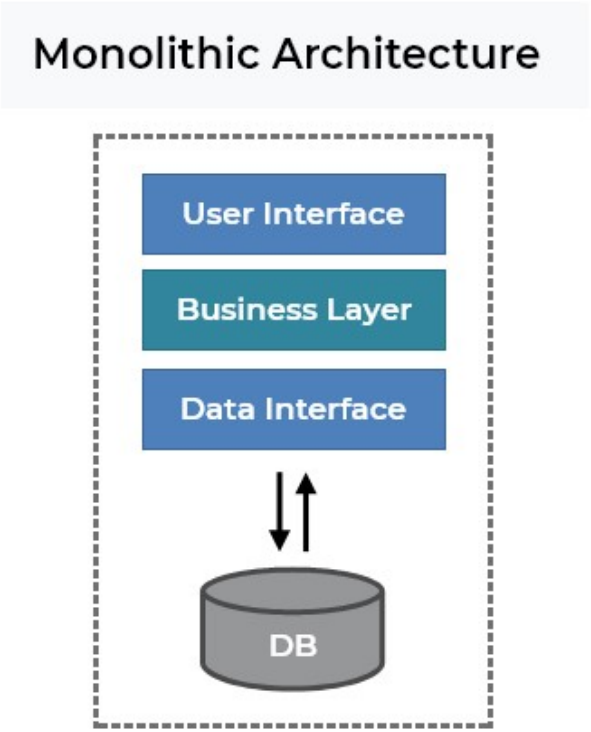


Монолитные приложения

Состоят из:

- базы данных
- клиентского пользовательского интерфейса
- серверного приложения

Все части ПО унифицированы, все функции управляются в одном месте.



Плюсы монолитной архитектуры

- Консистентность системы за счет использования локальных транзакций.
- Нет затрат на взаимодействие по сети, за счет этого лучшая производительность.
- Упрощенная разработка и развертывание.
- Для функционирования требуется меньше серверов.

Минусы монолитной архитектуры

- С ростом проекта увеличивается сложность приложения.
- Требуются мощные аппаратные ресурсы для поддержания необходимой производительности.
- Сложность рефакторинга и внедрения новых технологий.
- Любые изменения требуют полного повторного развертывания.

Плюсы микросервисной архитектуры

- Писать и поддерживать небольшие сервисы проще, чем большие.
- Практически из коробки получаем горизонтально масштабируемый и отказоустойчивый код.
- Упрощается тестирование, т.к. требуется покрыть тестами лишь API, при этом замокав ответы внешних систем.
- Высокая стабильность.
- Разнообразие технологий.
- Меньшая стоимость ошибок.
- Значительно упрощается обновление системы, т.к. для добавления функциональности требуется обновить лишь часть системы.

Минусы микросервисной архитектуры

- Система должна работать быстро, т.к. теперь ко времени выполнения самих операций требуется прибавлять время взаимодействия по сети.
- Нужно иметь хорошую систему деплоя и развертывания новых виртуальных машин.
- Требуется хорошее описание внешнего API.

Минусы микросервисной архитектуры

- С появлением большого количества сетевого трафика накладывается большая ответственность на сеть и отказоустойчивость оборудования.
- В связи с этим нужно при проектировании программы обязательно обрабатывать недоступность и ошибки от внешней системы.
- Часто становится очень трудно разбить систему на сервисы из-за сильной связанности данных.

Как понять что у нас микросервисы?

- Маленькие.
- Сфокусированные, т.е. решают только одну бизнес-задачу.
- Слабосвязанные, т.е. изменение одного сервиса не требует изменений в другом.
- Высокосогласованные, т.е. содержат все нужные методы решения поставленной задачи.

Распределенная система

- Взаимодействует несколько независимых систем.
- Никто никому ничем не обязан.
- Нет достоверной информации о состоянии других участников системы.
- При этом вся система должна всегда функционировать.

Монолит vs. Микросервисы

Монолит	Микросервисы
MVP, Proof Of Concept.	Есть четкое понимание *как* будет развиваться приложение, какая функциональность потребуется в дальнейшем.
Нет понимания в какую сторону будет расти приложение.	Есть понимание какая будет нагрузка на приложение, какие части системы будут наиболее нагружены.
Разработка ведется силами маленькой команды, нет достаточной компетенции в DevOps и других процессах.	Какие-то части системы делаются как *временное* решение, которые будут заменены в дальнейшем.
Размер приложения небольшой, но критически важна консистентность данных.	Планируется хранение большого объема данных.